

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen

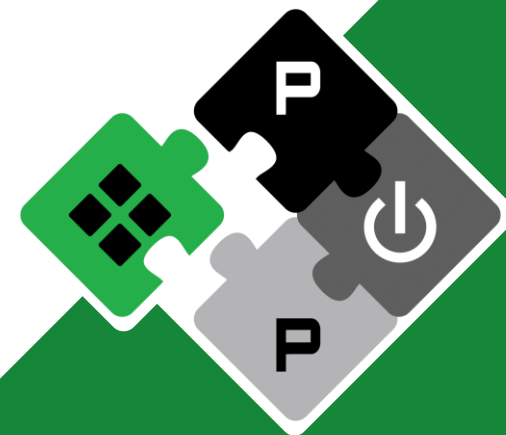
zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli
Luca Benini

avanelli@iis.ee.ethz.ch
lbenini@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!



@pulp_platform



pulp-platform.org



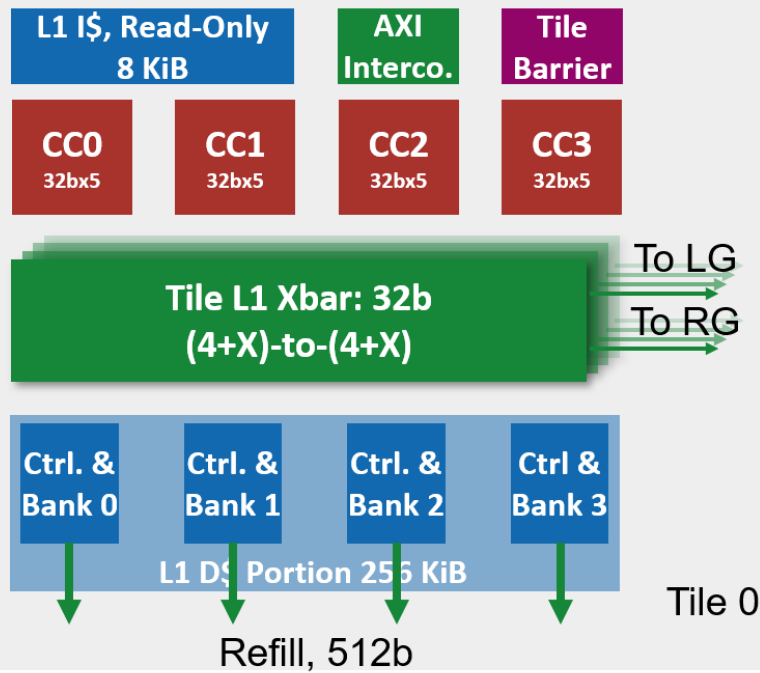
youtube.com/pulp_platform



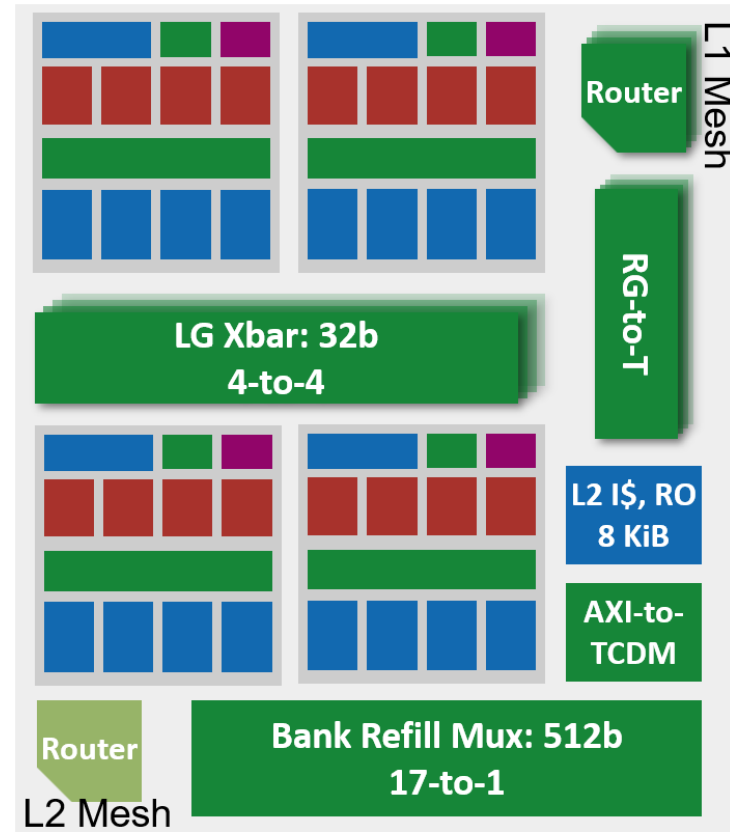
Hardware Development (Cluster)



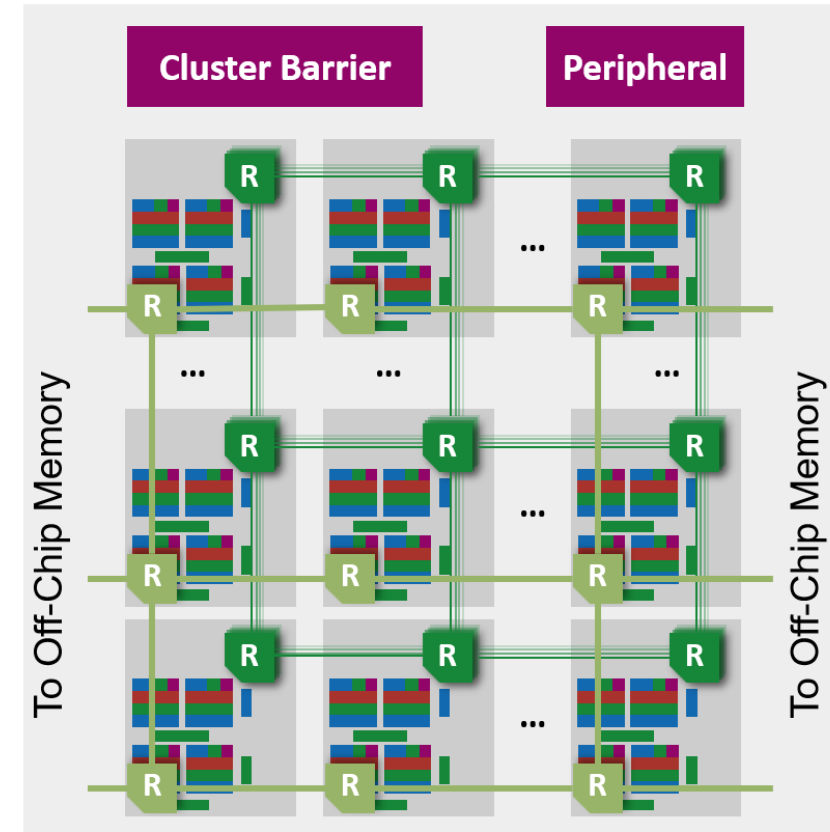
Tile



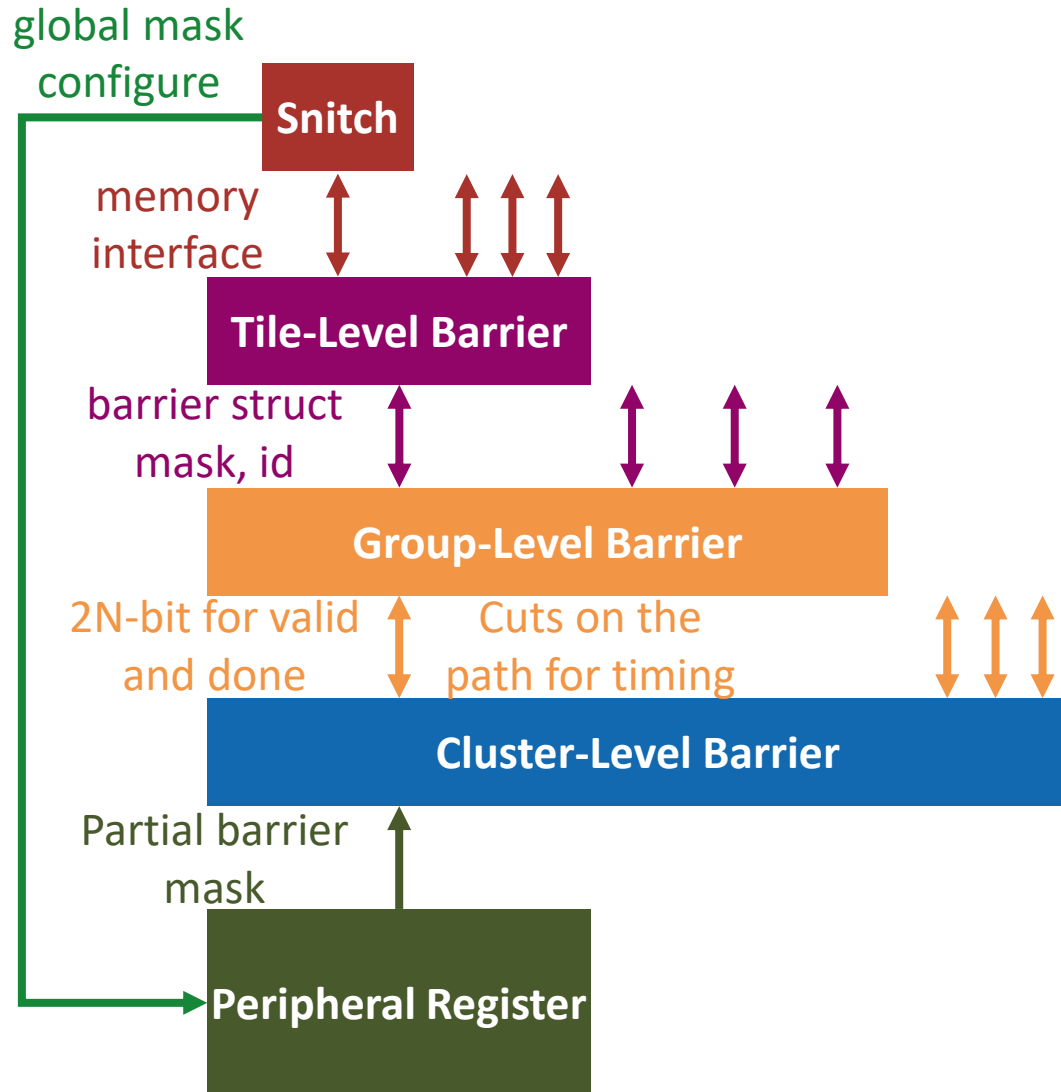
Group



Cluster



Hardware Development (Cluster)



• Multi/Partial HW Barrier Support

- Tile, Group, Cluster level hardware barrier
 - Tree-like structure
 - Support local partial barrier (do not need to sync with next level)
- Multiple barrier slots
 - Default 2 slots for normal configurations
 - Default 4 slots for dual-scalar configurations
 - Changeable in configuration files
- To use:
 - Host0/1 barrier:
`snrt_cluster_host0/1_barrier(<BID>)`
 - Full barrier: `snrt_cluster_hw_barrier()`
 - More customized configurations see runtime [README.md](#)



Hardware Development (Cluster)



- **Plans**

- Benchmarking and performance tuning
- Some nice to have features

- **DSE support for folded cache (WIP)**

DSE is used link different ways to provide more MSHR subentries for same address miss. This feature was turned off for folded cache support. We have discovered it limits our miss handling in real kernels. (need to have re-entry mechanism from the paper)

- **Group-level partitioning (Planned)**

Currently only support tile-level partitioning. Nice to have more options here for different task sizes

- **More L2 channel support (Planned)**

Support both 8 and 16 channels for off-chip connection. (TeraPool uses 16 channels)

- **Better dual-scalar core division (Idea)**

Current methods need SW written lock-unlock syncs. Nice to have a hardware-driven scheduling. This may need multi-thread support in Vector core (may not have enough time to implement in project scope)



Backend timing optimization for Insitu cache



- **Run the group-level with the timing optimized insitu cache**
 - No change in RTL behavior, only logic level optimization
 - Use the same floorplan
 - The placement is finished (can check setup timing), cts is running
- **Post-placement results**

	Baseline	Optimized	Reduced
WNS	-0.237ns	-0.147ns	-38%
TNS	-269.1ns	-14.6ns	-94.6%
Violating endpoints	6325	2190	-65.4%
Insitu cache ctrl area / bank	20548 μm^2	19262 μm^2	-6.3%

- **The critical path is no longer in the insitu cache:**
 - Baseline (inside the \$ ctrl): meta-SRAM Q $\rightarrow$ decoder $\rightarrow$ Cache_FSM $\rightarrow$ data-bank we $\rightarrow$ skew-column grant $\rightarrow$ meta SRAM clock gate
 - Optimized (snitch instr fetch): snitch next PC $\rightarrow$ icache fetch $\rightarrow$ instr retire clock gate



RLC kernel development



- **Data-Locality Optimizations**

- To reduce **remote NoC accesses** and **shared-cache contention**
- Utilize the private partition
 - Config the tile cache to be **2 private + 2 shared partition**
 - Keep RLC state/queues in the **tile-private L1**
 - Execute UE pkg processing on local cores
- Keep the linked list nodes descriptors contiguously mapped, the payload is still randomly mapped
- Hierarchical work scheduling
 - replace global shared work queue with global → per-group work queue.



RLC test cases results



- **We benchmark the latest RLC kernel with the 3 test cases on GVSoC model**
 - HW config:
 - 4x4 mesh; 16 groups $\times$ 4 tiles $\times$ 4 cc (1 Snitch + Spatz) = 256 cc
 - L1 \$: per tile 4 banks $\times$ (4 ways $\times$ 256 sets $\times$ 64 B) = 256 KiB, 16 MiB total
 - Memory: 64GB/s x8, 50-cycle latency

Throughput	SW config	Target	Measured	Compare to target
Test case 1 (single user peak)	16 Producers + 48 Consumers	20Gbps (1.85 M pkt/s)	1.83 M pkt/s	-1.0%
Test case 1 (multi-user peak)	48 Producers + 144 Consumers	50Gbps (7.81 M pkt/s)	9.29 M pkt/s	+19.0%
Test case 3 (multi-user average)	48 Producers + 144 Consumers	20Gbps (3.12 M pkt/s)	3.22 M pkt/s	+2.6%

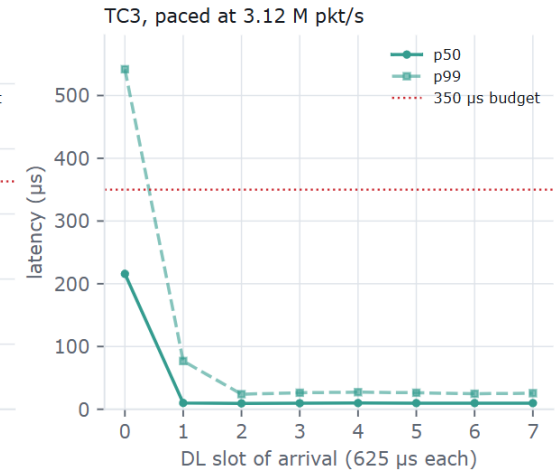
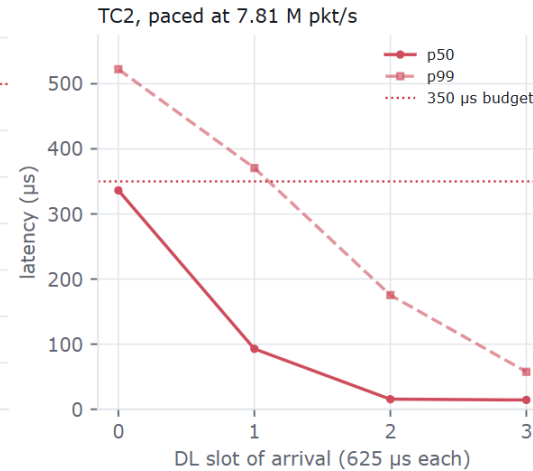
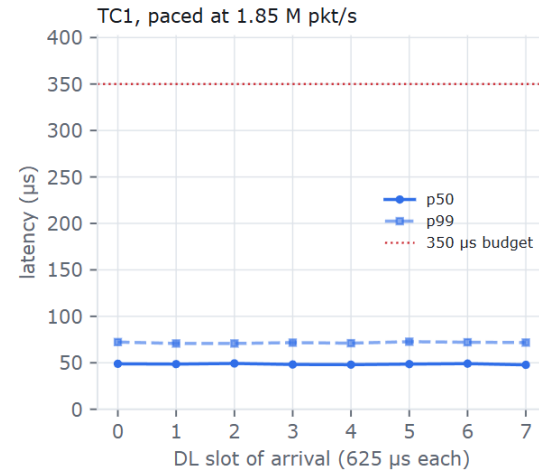


RLC test cases results

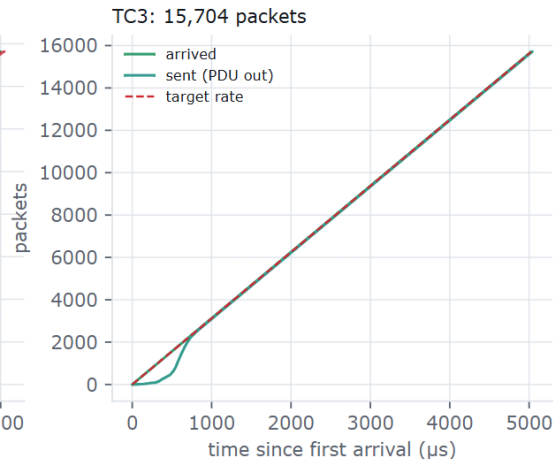
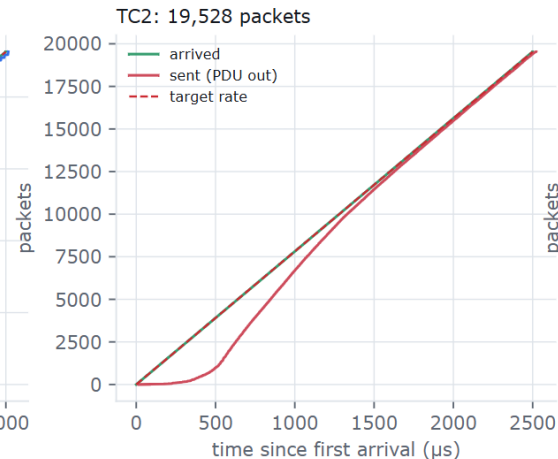
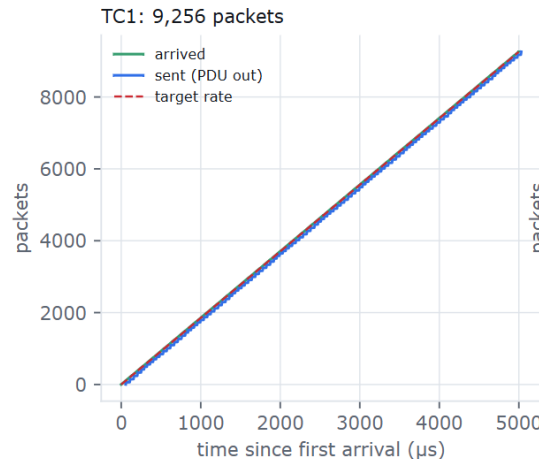


- **Latency**

- Latency per slot
 - Initial cache miss and cache contention at the start



- Pkg in and out against the target rate



Next Step

- **DATE 2027**
 - Paper submitted
 - Results: 23 November 2026 AoE
- **Diyoun Side**
 - Performance collection and optional features
- **Zexin Side**
 - GVSoC development, fix the dramsys under the cachepool_v3 target
 - SW development based on the benchmark plan
 - Finish the cts and routing of the group backend implementation



Timeline



		2025					2026							
		11	12	1	2	3	4	5	6	7	8	9	10	11
WP1 ManyCore Architecture	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] Complete													
	Interco. Optimization (multi-group) [RTL] Almost Complete													
	Timing optimization for NoC [RTL] Ongoing by colleague													
	NoC Topology Exploration [RTL, GVSoC] Ongoing by student													
WP2 Cache	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL] Complete													
	Folded Cache [GVSoC + RTL] Complete													



Timeline



	2025								2026						
		11	12	1	2	3	4	5	6	7	8	9	10	11	
WP3 VPE	Shared vector core [RTL + GVSoC] Almost Complete														
WP4 Benchmark	Single-user kernel optimization [SW] Ongoing														
	Multi-user and more scenarios [SW] Ongoing														
Report	Final Report and Benchmark														

L1 Access Latency



- **Problem: increased L1 access latency when scaling up**
- **Solutions**
 - **(Complete, RTL & Physical) Hardware optimization**
Further optimize the interconnection and cache controller to reduce access latency on existing architecture
=> Retiming the current interconnection, remove unnecessary cuts in interco and cache
=> Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
Currently has reduced to 6 cycles:
Core => Xbar => AMO => Ctrl (2) => AMO => Core
 - **(Complete, RTL) Cache partitioning**
Partition the shared L1 to keep data localized to avoid remote access latency
 - **(Complete with some problems, RTL & Physical) WT-L1**
Explore and evaluate a private WT L1 for each core
=> Performance degrade on single tile
=> Needs more work to support larger system



L1 Access Latency



- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
 - **(Ongoing, Physical) Reduce hop latency**
Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
=> Reduce the latency by 1/2
 - **(Planned, Physical) Cut clk-domain (optional)**
If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
=> Effectively cut the latency by 1/3
 - **(Planned, GVSoC & Physical) NoC Topology**
Explore different NoC topologies
=> Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2



PE-Related



- **Problem: Improve scalar core performance**
- **Solutions**
 - **(Complete, RTL & GVSOC) Shared vector core**
Explore two Snitch sharing one Spatz configuration to improve scalar performance
Software complexity needs to be considered, especially on the possible conflicting use of register file.
Discussed Solution: critical section



RLC Workload



- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
 - **(Ongoing, SW) Kernel optimization**
Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
 - **(Planned, SW) Kernel development**
Implement the missing use cases for multi-user



Thank you!

Q&A

